Setup

LegalDataPlatform

Humberto Henriquez — Senior Data Engineer / Solutions Architect
Generado 2026-04-24

Setup — cómo correr el proyecto en cualquier máquina

Esta guía es para alguien que acaba de recibir el proyecto y nunca lo ha corrido. Está escrita para Windows, macOS y Linux.

TL;DR — un solo comando

Si tienes los prerequisitos instalados:

Windows (PowerShell)

```
cd ruta\al\proyecto
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1
```

macOS / Linux / WSL / Git Bash

```
cd /ruta/al/proyecto
bash scripts/setup.sh
```

El script se encarga de: verificar prerequisitos, crear `.env`, crear el virtualenv de Python, instalar dependencias, levantar Docker Compose, esperar a que PostgreSQL esté listo, correr migraciones y generar datos de ejemplo.

Prerequisitos (instalar antes de correr el script)

Herramienta	Versión mínima	Link
Python	3.11+	https://www.python.org/downloads/
Docker Desktop	24+	https://www.docker.com/products/docker-desktop
Git	cualquiera reciente	https://git-scm.com/downloads

Verifica que están instalados

```
python --version     # debe ser ≥ 3.11
docker --version
docker compose version
git --version
```

Si alguno falta o la versión es vieja, instálalo antes de continuar.

Paso cero: ubicarte en la carpeta del proyecto

El error más común en Windows CMD: `cd E:\Proyectos\ ...` no cambia de unidad por defecto.

Windows CMD

```
cd /d E:\Proyectos\LegalDataPlatform
```

El flag `/d` hace que `cd` también cambie de unidad.

Windows PowerShell (recomendado)

```
cd E:\Proyectos\LegalDataPlatform
```

PowerShell sí cambia de unidad sin flag.

macOS / Linux / WSL / Git Bash

```
cd ~/ruta/al/proyecto/LegalDataPlatform
```

Setup paso a paso (sin script)

Si prefieres ejecutar los pasos manualmente:

1. Crear `.env`

```
# Unix / Git Bash / PowerShell
cp .env.example .env

# Windows CMD
copy .env.example .env
```

2. Crear y activar virtualenv

```
python -m venv .venv

# Activar:
source .venv/bin/activate           # macOS / Linux / WSL / Git Bash
.venv\Scripts\activate              # Windows cmd / PowerShell
```

3. Instalar dependencias

```
pip install --upgrade pip
pip install -e ".[dev]"
```

4. Levantar Docker

```
docker compose up -d
docker compose ps     # verifica que todos estén "Up"
```

5. Aplicar migraciones

```
alembic upgrade head
```

6. Generar datos de ejemplo

```
python scripts/seed_data.py
```

7. Correr el pipeline

```
make pipeline
# o, sin Make:
python -m src.pipelines.orchestration.legal_ingestion_flow
```

Puertos que necesita el proyecto

Si tienes otras cosas corriendo localmente, estos puertos deben estar libres:

Puerto	Servicio
5432	PostgreSQL
6432	PgBouncer
9000	MinIO S3 API
9001	MinIO Console
4200	Prefect UI
9090	Prometheus
3000	Grafana

Si alguno está ocupado, edita `docker-compose.yml` y cambia el mapping externo (`host:container` → cambia el host).

Diferencias por sistema operativo

Windows

- Usa **PowerShell** o **Git Bash** en vez de CMD. CMD no entiende comandos Unix (`cp` , `source`) que aparecen en la documentación.
- Si ves problemas con saltos de línea en scripts `.sh` , configura git para que no convierta line endings: `git config --global core.autocrlf input` .
- Docker Desktop debe estar corriendo antes de cualquier comando `docker` .

macOS

- Si `python` apunta a Python 2, usa `python3` explícitamente o crea un alias.
- Si usas Apple Silicon (M1/M2/M3), Docker descargará imágenes ARM64 automáticamente. Todas las imágenes del stack (postgres, minio, prefect, grafana) tienen build ARM64.

Linux

- Puede que necesites agregar tu usuario al grupo `docker` para correr sin `sudo` : `sudo usermod -aG docker $USER` y re-loguearte.
- En distros minimalistas puede faltar `build-essential` / `python3-dev` para compilar algunas dependencias: `sudo apt install build-essential python3-dev libpq-dev` .

Troubleshooting

"cd E:... no funciona" (Windows CMD)

Usa `cd /d E:\...` o cambia a PowerShell.

"docker: command not found"

Docker Desktop no está instalado o no está en el PATH. Instala Docker Desktop y reinicia la terminal.

"Cannot connect to the Docker daemon"

Docker Desktop no está corriendo. Ábrelo desde el menú/Aplicaciones y espera a que el ícono esté estable.

"port 5432 already in use"

Ya tienes un PostgreSQL local corriendo. Opciones:
- Detenerlo: `brew services stop postgresql` / `sudo service postgresql stop` / detenerlo desde Docker Desktop.
- Cambiar el puerto en `docker-compose.yml`: cambia `"5432:5432"` a `"5433:5432"` y actualiza `POSTGRES_PORT=5433` en `.env`.

"ModuleNotFoundError: No module named 'src'"

El virtualenv no está activo o falta `pip install -e ".[dev]"`. Verifica con `which python` (debe apuntar a `.venv/bin/python` o `.venv\Scripts\python.exe`).

"ExecutionPolicy" al correr setup.ps1

Windows bloquea scripts PowerShell por defecto. Usa:

```
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1
```

El script pide contraseña para `alembic` / `pip`

Probablemente estás corriendo con el Python del sistema, no el del virtualenv. Activa el venv primero.

Generación de PDFs falla con "No Chromium-based browser found"

El script busca Chrome/Edge/Chromium automáticamente. Si lo tienes en una ubicación no estándar, setea:

```
export LDP_BROWSER=/ruta/a/chrome          # Unix
$env:LDP_BROWSER = "C:\ruta\a\chrome.exe"  # PowerShell
```

Limpiar todo y empezar de cero

```
docker compose down -v    # detiene servicios y borra volúmenes (datos)
rm -rf .venv              # borra virtualenv (Unix)
rmdir /s .venv            # Windows CMD
Remove-Item -Recurse -Force .venv  # PowerShell
rm -rf data/samples       # borra datos generados
```

Luego vuelve a correr el script de setup.